

Constrained optimization of a convex quadratic function using an Active-set method

Marco Barberio

Federico Giusti

Master Degree in Computer Science.

m.barberio2@studenti.unipi.it, f.giusti29@studenti.unipi.it.

Computational Mathematics, Academic Year: 2025/2026

Group 20 – Project 2 (NO-ML)



Contents

1	Introduction	2
1.1	Notation	2
2	Background	2
2.1	Objective function	2
2.2	Constraints and Geometry of the Feasible Set	3
2.3	Compact Form of the Equality Constraints	3
2.4	Constraint Functions and Their Gradients	4
2.5	Karush-Kuhn-Tucker conditions	5
3	Active-set method	6
3.1	Active and free sets	6
3.2	Subproblem	7
3.3	Null-space solution of the subproblem	8
3.3.1	How we build D and x_p in practice	14
3.4	Feasibility step	15
3.5	Optimality step	19
3.6	The proposed active-set algorithm	21
3.7	Convergence analysis	23
3.8	Computational complexity	25
4	Experimental Results	27
4.1	Off-the-shelf solver	28
4.2	Scalability with respect to the problem dimension	29
4.3	Conditioning robustness	32
4.4	SPD vs SPSD	35
4.5	Sparsity robustness	38

1 Introduction

In this project, our goal is to develop an algorithm belonging to the class of active-set methods to solve the following constrained minimization problem:

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + q^T x : \sum_{i \in I^k} x_i = 1 \quad k \in K, \quad x \geq 0 \right\} \quad (1)$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \dots, n\}$ (i.e., $\bigcup_{k \in K} I^k = \{1, \dots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ positive semidefinite matrix.

1.1 Notation

The following notation will be adopted through the report:

We denote by n the dimension of the decision variable $x \in \mathbb{R}^n$. The set K indexes the blocks of variables, and $|K|$ denotes its cardinality.

Δ_m denotes the standard simplex of m points. The vector $\mathbf{1}_{|K|}$ denotes the vector in $\mathbb{R}^{|K|}$ whose components are all equal to one. We denote by e_j the j -th canonical vector of $\mathbb{R}^n$. The symbols $\nabla f(x)$ and $\nabla^2 f(x)$ denote the gradient and the Hessian matrix of f , respectively.

2 Background

In this section, we report the key properties of the objective function and the constraints that are relevant for the analysis.

2.1 Objective function

The objective function of problem (P) is given by

$$f(x) = x^T Q x + q^T x,$$

where $Q \in \mathbb{R}^{n \times n}$ is a symmetric positive semidefinite matrix and $q \in \mathbb{R}^n$.

Since Q is positive semidefinite, the quadratic term $x^T Q x$ is convex. The linear term $q^T x$ is also convex. Therefore, the objective function $f(x)$ is convex.

The gradient of f is given by

$$\nabla f(x) = 2Qx + q,$$

and the Hessian matrix is

$$\nabla^2 f(x) = 2Q.$$

2.2 Constraints and Geometry of the Feasible Set

Each group of variables associated with a set of indices I^k defines a simplex.

Proposition 1. *The standard simplex in $\mathbb{R}^m$ is the convex hull of the canonical basis $\{e_1, \dots, e_m\}$, that is,*

$$\Delta_m = \text{conv}\{e_1, \dots, e_m\}.$$

Since a simplex is a polytope, it is a convex set.

The feasible domain can be written as the intersection of K simplices:

$$\mathcal{D} = \bigcap_{k \in K} \left\{ x \in \mathbb{R}^n : \sum_{i \in I^k} x_i = 1, x_i \geq 0 \right\}. \quad (2)$$

Proposition 2. *The intersection of convex sets is convex [1].*

Hence the feasible domain $\mathcal{D}$ is convex.

2.3 Compact Form of the Equality Constraints

The equality constraints

$$\sum_{i \in I^k} x_i = 1 \quad \forall k \in K \quad (3)$$

can be written in compact form as

$$Ax = b, \quad (4)$$

where $A \in \mathbb{R}^{|K| \times n}$ and $b \in \mathbb{R}^{|K|}$.

The matrix A is defined componentwise as

$$a_{ki} = \begin{cases} 1 & \text{if } i \in I^k, \\ 0 & \text{otherwise} \end{cases} \quad k \in K, i = 1, \dots, n. \quad (5)$$

Since the sets $\{I^k\}_{k \in K}$ are pairwise disjoint, each index i belongs to exactly one set I^k . Consequently, each column of A contains exactly one entry equal to 1, while all the other entries in that column are zero. Moreover,

$$b = \mathbf{1}_{|K|}, \quad (6)$$

Proposition 3. *The matrix A has full row rank, i.e.,*

$$\text{rank}(A) = |K|.$$

Proof. The k -th row of A is the indicator vector of the set I^k . Since the sets $\{I^k\}_{k \in K}$ are pairwise disjoint, the supports of the rows of A are disjoint. Therefore, no row can be written as a linear combination of the others, and the rows are linearly independent. Hence, $\text{rank}(A) = |K|$. $\square$

2.4 Constraint Functions and Their Gradients

The equality constraints can be written as a family of functions

$$h_k(x) = 0 \quad \forall k \in K, \quad (7)$$

where

$$h_k(x) := \sum_{i \in I^k} x_i - 1. \quad (8)$$

The non-negativity constraints can be written as

$$g_j(x) \leq 0 \quad \forall j = 1, \dots, n, \quad (9)$$

where

$$g_j(x) := -x_j. \quad (10)$$

For each $k \in K$, the gradient of h_k satisfies

$$(\nabla h_k(x))_j = \begin{cases} 1 & \text{if } j \in I^k, \\ 0 & \text{otherwise,} \end{cases} \quad j = 1, \dots, n. \quad (11)$$

Therefore, $\nabla h_k(x) \in \mathbb{R}^n$ is exactly the indicator vector of the set I^k .

Corollary 1. *The k -th row of the matrix A coincides with the gradient $\nabla h_k(x)$.*

Finally, for each $j = 1, \dots, n$, the gradient of g_j is

$$\nabla g_j(x) = -e_j, \quad (12)$$

where e_j denotes the j -th canonical vector of $\mathbb{R}^n$.

2.5 Karush-Kuhn-Tucker conditions

Let λ_k be the Lagrange multipliers associated with the equality constraints $h_j(x) = 0$, and let $\mu_i \geq 0$ be the multipliers associated with the inequality constraints $x_i \geq 0$.

The Lagrangian function is:

$$\mathcal{L}(x, \lambda, \mu) = x^T Q x + q^T x + \sum_{k \in K} \lambda_k \left(\sum_{i \in I^k} x_i - 1 \right) - \sum_{i=1}^n \mu_i x_i. \quad (13)$$

Since the objective function is convex and the constraints are linear, the KKT conditions are necessary and sufficient for optimality.

Moreover, a point (x^*, λ^*, μ^*) is optimal if and only if the following properties are satisfied:

- Primal feasibility:

$$\sum_{i \in I^k} x_i^* = 1 \quad \forall k \in K, \quad x_i^* \geq 0 \quad \forall i. \quad (14a)$$

- Dual feasibility:

$$\mu_i^* \geq 0 \quad \forall i. \quad (14b)$$

- Stationarity:

$$\nabla f(x^*) + \sum_{k \in K} \lambda_k^* a^k - \mu^* = 0. \quad (14c)$$

- Complementary slackness:

$$\mu_i^* x_i^* = 0 \quad \forall i. \quad (14d)$$

The condition (14d) characterizes the active set at the solution. For each index, either $x_i^* = 0$ or $\mu_i^* = 0$.

When $x_i^* = 0$, the corresponding inequality constraint is active, meaning that the solution lies on the boundary of the feasible region with respect to that variable. In this case, the multiplier μ_i^* satisfies $\mu_i^* \geq 0$, indicating that the constraint contributes to the stationarity condition. In the complementary case, where $x_i^* > 0$, the variable lies in the interior of the feasible region, therefore the constraint is inactive and its multiplier must be zero.

3 Active-set method

In this section we develop an active-set method for solving problem (P) . Since the feasible set is defined by block equality constraints together with nonnegativity constraints, the method distinguishes at each iteration between active and free variables.

Given a feasible iterate, the variables in the active set are fixed to zero, and the problem is restricted to the free variables. This yields an equality-constrained quadratic subproblem, which is solved through a null-space parametrization of the equality constraints. The resulting point is then used either to update the iterate or to revise the active set, until the KKT conditions are satisfied.

3.1 Active and free sets

Since the only inequality constraints are the bound constraints $x_i \geq 0$, the active set at a feasible point x is naturally defined as

$$\mathcal{A}(x) := \{i \in \{1, \dots, n\} : x_i = 0\},$$

and the corresponding free set is

$$\mathcal{F}(x) := \{1, \dots, n\} \setminus \mathcal{A}(x).$$

Intuitively, indices in $\mathcal{A}(x)$ identify the face of the feasible polytope on which the current iterate lies.

3.2 Subproblem

Given a point $\bar{x}$, its corresponding active set

$$B = \mathcal{A}(\bar{x}),$$

and its corresponding free set

$$F = \mathcal{F}(\bar{x}),$$

the first step consists in restricting the problem by explicitly enforcing the constraints associated with the active set.

Since the bound constraints associated with the active indices imply

$$x_i = 0 \quad \forall i \in B,$$

the variables indexed by B are fixed and therefore no longer need to be treated as optimization variables. As a result, the problem can be reduced and formulated only in terms of the free variables x_F .

Denoting by Q_{FF} the submatrix of Q obtained by selecting the rows and columns indexed by F , and by q_F the subvector of q corresponding to the same indices, the reduced problem becomes

$$\min \left\{ x_F^T Q_{FF} x_F + q_F^T x_F : \sum_{i \in I^k \cap F} x_i = 1 \quad \forall k \in K \right\}. \quad (15)$$

In other words, the variables belonging to the active set are fixed to zero, while the remaining variables must satisfy the equality constraints associated with each block.

This reduced problem corresponds to the equality-constrained quadratic subproblem associated with the current active set. Its solution provides a candidate point $\bar{x}$ for the next iteration of the active-set method.

3.3 Null-space solution of the subproblem

The saddle-point system introduced in the previous subsection is symmetric but indefinite. In order to obtain a numerically more stable and scalable formulation, we exploit the special block structure of the equality constraints and apply a null-space reduction.

For each block k such that $I^k \cap F \neq \emptyset$, choose one index $p(k) \in I^k \cap F$. The equality constraint associated with block k can be written as

$$x_{p(k)} = 1 - \sum_{i \in I^k \cap F \setminus \{p(k)\}} x_i.$$

Hence, within each block, one variable is determined by the others. The total number of independent variables is therefore $|F| - |K|$.

Let $z \in \mathbb{R}^{|F|-|K|}$ denote the vector collecting all independent variables (i.e., all free variables except the pivot ones).

All feasible vectors satisfying the block constraints can therefore be parametrized as

$$x_F = x_p + Dz,$$

where x_p is a particular feasible vector satisfying the equality constraints and the columns of D span the null space of the constraint matrix.

Substituting this parametrization into the objective function yields

$$f(x_F) = f(x_p + Dz) = (x_p + Dz)^T Q_{FF} (x_p + Dz) + q_F^T (x_p + Dz).$$

This substitution can be interpreted as a change of variables that parametrizes the feasible set in terms of the independent variables z . Therefore, the objective function can be expressed as a function of z .

Expanding the terms and collecting quadratic and linear terms in z gives

$$f(z) = z^T D^T Q_{FF} D z + z^T (2D^T Q_{FF} x_p + D^T q_F) + c,$$

where c is a constant independent of z . Since constant terms do not affect the minimizer, they can be discarded. The reduced problem therefore takes the standard

quadratic form

$$\min_z \left\{ \frac{1}{2} z^T H z + g^T z \right\}, \quad (16)$$

where

$$H = 2D^T Q_{FF} D, \quad g = 2D^T Q_{FF} x_p + D^T q_F.$$

Since $Q \succeq 0$, we have

$$z^T H z = 2(Dz)^T Q_{FF} (Dz) \geq 0,$$

so that H is symmetric positive semidefinite.

The first-order optimality condition is obtained by setting the gradient equal to zero, which yields

$$H z = -g. \quad (17)$$

In order to solve this system, we first attempt a Cholesky factorization of H . If H is positive definite, the factorization succeeds, the system admits a unique solution, and it can be efficiently solved by means of the Cholesky factorization.

If H is singular, the Cholesky factorization fails, and we instead use the Singular Value Decomposition in order to compute the pseudoinverse H^+ . In this case, two situations may arise. If the system is compatible, then it admits infinitely many solutions, and using the SVD we compute the minimum-norm solution $\bar{z} = -H^+ g$. To determine whether the system is compatible, we compute the residual

$$r = \|H\bar{z} + g\|.$$

If the residual is below a given tolerance, the solution is accepted and we set $z = \bar{z}$.

Otherwise, the system is incompatible, which means that $-g \notin \text{range}(H)$. In this case, the reduced quadratic problem is unbounded below along a direction in the null space of H . Therefore, we compute a direction $p \in \ker(H)$ such that

$$g^T p < 0,$$

and map it back to the original variable space through

$$d = Dp.$$

Proposition 4. *Let $A \in \mathbb{R}^{n \times n}$ be a symmetric positive semidefinite matrix, and consider the unconstrained quadratic minimization problem*

$$\min_{x \in \mathbb{R}^n} f(x), \quad f(x) = \frac{1}{2}x^T A x + b^T x.$$

Consider the gradient of f

$$\nabla f(x) = Ax + b,$$

the first-order optimality condition is the linear system

$$Ax = -b.$$

Assume that this system is incompatible, namely

$$-b \notin \text{range}(A).$$

Then there exists a vector $p \in \ker(A)$ such that

$$p \neq 0, \quad b^T p < 0.$$

Consequently, the problem $\min_x f(x)$ is unbounded below.

Proof. Since A is symmetric, we have

$$\text{range}(A) = \ker(A)^\perp.$$

Because $-b \notin \text{range}(A)$, it follows that $b \notin \text{range}(A)$. Hence the orthogonal projection of b onto $\ker(A)$ is nonzero.

Let

$$b = b_R + b_N, \quad b_R \in \text{range}(A), \quad b_N \in \ker(A), \quad b_N \neq 0.$$

Define

$$p := -b_N.$$

Then $p \in \ker(A)$ and $p \neq 0$. Since $Ap = 0$, we obtain

$$p^T A p = 0.$$

Moreover, using the orthogonality between $\text{range}(A)$ and $\ker(A)$,

$$b^T p = (b_R + b_N)^T (-b_N) = -b_R^T b_N - \|b_N\|^2 = -\|b_N\|^2 < 0.$$

Therefore, for every $\alpha \geq 0$,

$$f(x + \alpha p) = \frac{1}{2}(x + \alpha p)^T A(x + \alpha p) + b^T(x + \alpha p).$$

Expanding the quadratic term, we obtain

$$(x + \alpha p)^T A(x + \alpha p) = x^T A x + \alpha x^T A p + \alpha p^T A x + \alpha^2 p^T A p.$$

Since A is symmetric, we have

$$x^T A p = p^T A x,$$

and therefore

$$(x + \alpha p)^T A(x + \alpha p) = x^T A x + 2\alpha x^T A p + \alpha^2 p^T A p.$$

Moreover, the linear term becomes

$$b^T(x + \alpha p) = b^T x + \alpha b^T p.$$

Substituting these expressions into $f(x + \alpha p)$ yields

$$f(x + \alpha p) = \frac{1}{2}x^T A x + \alpha x^T A p + \frac{1}{2}\alpha^2 p^T A p + b^T x + \alpha b^T p.$$

Now, since $p \in \ker(A)$, we have $Ap = 0$. Hence

$$x^T A p = x^T 0 = 0, \quad p^T A p = p^T 0 = 0.$$

Therefore,

$$f(x + \alpha p) = \frac{1}{2}x^T A x + b^T x + \alpha b^T p.$$

we finally obtain

$$f(x + \alpha p) = f(x) + \alpha b^T p.$$

Because $b^T p < 0$, we conclude that

$$f(x + \alpha p) \rightarrow -\infty \quad \text{as } \alpha \rightarrow +\infty.$$

Hence the quadratic problem is unbounded below. □

We choose the direction

$$p := -r.$$

Proposition 5. *Let*

$$f(z) = \frac{1}{2}z^T H z + g^T z,$$

with $H \in \mathbb{R}^{m \times m}$ symmetric positive semidefinite, and let

$$\bar{z} = -H^+ g$$

be the minimum-norm least-squares solution associated with the system

$$H z = -g.$$

Define the residual vector

$$r := H \bar{z} + g.$$

Then:

1. $r \in \ker(H)$;
2. *if $r \neq 0$, then the vector*

$$p := -r$$

is a strict descent direction for f , namely

$$g^T p < 0.$$

Proof. Since $\bar{z} = -H^+ g$, we have

$$r = H \bar{z} + g = -H H^+ g + g.$$

Therefore

$$H r = -H^2 H^+ g + H g.$$

Using the identity

$$H H^+ H = H,$$

and multiplying it on the left by H , we obtain

$$H^2 H^+ = H.$$

Hence

$$Hr = -Hg + Hg = 0,$$

which proves that $r \in \ker(H)$.

Now, since

$$r = H\bar{z} + g,$$

we can write

$$g = r - H\bar{z}.$$

Taking the scalar product with r , we obtain

$$g^T r = r^T r - r^T H\bar{z}.$$

Because H is symmetric and $Hr = 0$, it follows that

$$r^T H\bar{z} = (Hr)^T \bar{z} = 0.$$

Thus

$$g^T r = \|r\|^2.$$

If $r \neq 0$ and we define $p = -r$, then

$$g^T p = -g^T r = -\|r\|^2 < 0.$$

Therefore p is a strict descent direction. □

The algorithm then moves along this feasible descent direction while maintaining the non-negativity constraints. The step length is chosen as

$$\alpha = \min_{i:d_i < 0} \left(\frac{-x_i^t}{d_i} \right),$$

so that the new iterate

$$x^{t+1} = x^t + \alpha d$$

remains feasible and at least one component becomes zero. Consequently, a new constraint becomes active and the active set is updated.

Indeed, since $d_B = 0$, only free variables can decrease along the direction d . Hence, if $d_i < 0$, then $i \in F^t$, and therefore $x_i^t > 0$, because the current iterate x^t is feasible. It follows that

$$\frac{-x_i^t}{d_i} > 0.$$

Moreover, at least one index such that $d_i < 0$ must exist; otherwise the direction would remain feasible for arbitrarily large step lengths, contradicting the boundedness of the feasible set. Consequently, α is well defined and strictly positive.

If the solution z is obtained, the free variables are reconstructed through the parametrization

$$\bar{x} = x_F = x_p + Dz. \tag{18}$$

while the active variables remain fixed at zero. The resulting vector $\bar{x}$ provides a candidate minimizer of the current equality-constrained subproblem.

3.3.1 How we build D and x_p in practice

In our implementation, the null-space parametrization

$$x_F = x_p + Dz$$

is constructed explicitly by exploiting the block structure of the equality constraints.

We consider the subset of free indices

$$J_k = I^k \cap F.$$

If $J_k = \emptyset$, the block does not contribute to the reduced problem and is skipped. Otherwise, the first free index in the block is selected as pivot, namely

$$p(k) = J_k(1).$$

The particular feasible vector x_p is initialized to zero and, for every block with at least one free variable, the pivot component is set equal to one:

$$(x_p)_{p(k)} = 1.$$

All the other components remain zero. In this way, within each active block, x_p satisfies the corresponding equality constraint.

The matrix D is then constructed column by column. For every non-pivot free index $j \in J_k \setminus \{p(k)\}$, one column of D is defined as

$$D_{\cdot, \ell} = e_j - e_{p(k)},$$

where e_j and $e_{p(k)}$ are the canonical basis vectors associated with the free-variable indexing.

Therefore, increasing the variable indexed by j by one unit is compensated by decreasing the pivot variable by one unit, so that the sum over the block remains unchanged. As a consequence, each column of D belongs to the null space of the constraint matrix associated with the block equalities.

If a block contains $m_k = |J_k|$ free variables, then it contributes exactly $m_k - 1$ columns to D . Summing over all blocks, the total number of columns is

$$|F| - \#\{k : I^k \cap F \neq \emptyset\},$$

which is exactly the number of independent variables in the reduced problem.

With this construction, the parametrization

$$x_F = x_p + Dz$$

automatically satisfies all block equality constraints. Moreover, once the reduced solution z is computed, the vector of free variables can be reconstructed directly through the same formula.

3.4 Feasibility step

The candidate solution $\bar{x}$ may violate the non-negativity constraints $x \geq 0$.

To verify feasibility, we check whether

$$\bar{x}_i \geq 0 \quad \forall i.$$

If the candidate solution satisfies this condition, it is feasible and can be used to perform the optimality test.

Otherwise, if some components of $\bar{x}$ are negative, a step is taken from the current iterate x^t toward $\bar{x}$ while maintaining feasibility.

Let

$$d = \bar{x} - x^t \tag{19}$$

be the search direction.

Proposition 6. *Let x^t be the current feasible iterate, and let $\bar{x}$ be the solution of the equality-constrained quadratic subproblem associated with the current active set. If $\bar{x}$ is not feasible for problem (P) and $\bar{x} \neq x^t$, then the vector*

$$d = \bar{x} - x^t$$

is a descent direction for the objective function at x^t .

Proof. Since $\bar{x}$ minimizes the current equality-constrained subproblem and x^t is feasible for that subproblem, we have

$$f(\bar{x}) \leq f(x^t).$$

By convexity of f ,

$$f(\bar{x}) \geq f(x^t) + \nabla f(x^t)^T(\bar{x} - x^t).$$

Therefore,

$$\nabla f(x^t)^T(\bar{x} - x^t) \leq f(\bar{x}) - f(x^t) \leq 0.$$

If equality held, then $\bar{x}$ would not provide any improvement over x^t . Hence, in the case of interest where the algorithm generates a new candidate point $\bar{x} \neq x^t$, we obtain

$$\nabla f(x^t)^T d < 0,$$

and thus d is a descent direction. □

The step length α is chosen so that the first violated non-negativity constraint becomes active:

$$\alpha = \min_{i:d_i < 0} \frac{-x_i^t}{d_i}. \quad (20)$$

The next iterate is then defined as

$$x^{t+1} = x^t + \alpha d. \quad (21)$$

Proposition 7. *At each feasibility step, the steplength α satisfies*

$$0 < \alpha < 1.$$

Proof. Since $\bar{x}$ is infeasible, there exists at least one index j such that

$$\bar{x}_j < 0.$$

Because x^t is feasible, we have $x_j^t \geq 0$, and therefore

$$d_j = \bar{x}_j - x_j^t < 0.$$

Hence the set $\{i : d_i < 0\}$ is nonempty, so α is well defined.

We first prove that $\alpha > 0$. Let i be such that $d_i < 0$. We claim that $i \notin B^t$. Indeed, if $i \in B^t$, then by definition of the active set

$$x_i^t = 0.$$

Moreover, in the reduced subproblem the active variables are kept fixed at zero, so

$$\bar{x}_i = 0.$$

Thus

$$d_i = \bar{x}_i - x_i^t = 0,$$

which contradicts $d_i < 0$. Therefore every index with $d_i < 0$ must belong to the free set F^t .

Now, if $i \in F^t$, then by definition of $B^t = \{i : x_i^t = 0\}$ and feasibility of x^t ,

$$x_i^t > 0.$$

Since also $d_i < 0$, it follows that

$$\frac{-x_i^t}{d_i} > 0.$$

Taking the minimum over all i such that $d_i < 0$, we obtain

$$\alpha > 0.$$

We now prove that $\alpha < 1$. From the index j above, for which $\bar{x}_j < 0$, we have $d_j < 0$, so j is admissible in the definition of α . Hence

$$\alpha \leq \frac{-x_j^t}{d_j}.$$

It remains to show that the right-hand side is strictly smaller than 1. Since $d_j = \bar{x}_j - x_j^t$, we have

$$\frac{-x_j^t}{d_j} < 1 \iff -x_j^t > d_j \iff -x_j^t > \bar{x}_j - x_j^t \iff 0 > \bar{x}_j,$$

which is true by construction. Therefore

$$\frac{-x_j^t}{d_j} < 1,$$

and consequently

$$\alpha < 1.$$

Thus

$$0 < \alpha < 1.$$

□

By construction, x^{t+1} remains feasible with respect to the non-negativity constraints, that is,

$$x^{t+1} \geq 0.$$

Indeed, if $d_i \geq 0$, then $x_i^{t+1} = x_i^t + \alpha d_i \geq x_i^t \geq 0$; if $d_i < 0$, then

$$\alpha \leq \frac{-x_i^t}{d_i},$$

which implies

$$x_i^{t+1} = x_i^t + \alpha d_i \geq 0.$$

Moreover, for at least one index attaining the minimum, $x_i^{t+1} = 0$, so at least one non-negativity constraint becomes active.

3.5 Optimality step

Once the candidate solution $\bar{x}$ passed the feasibility test, the next step consists in verifying whether the current active set satisfies the Karush–Kuhn–Tucker optimality conditions.

From the stationarity condition

$$\nabla f(x) + A^T \lambda - \mu = 0,$$

and recalling that $\nabla f(x) = 2Qx + q$, we obtain

$$2Q\bar{x} + q + A^T \lambda - \mu = 0.$$

For indices in the free set we enforce

$$\mu_i = 0 \quad i \in F,$$

consistently with the complementary slackness condition.

Restricting the stationarity condition to the free variables yields

$$(A_F)^T \lambda = -(2Q\bar{x} + q)_F.$$

The multiplier vector λ can be obtained by solving the linear system

$$(A_F A_F^T) \lambda = -A_F (2Q\bar{x} + q)_F.$$

Proposition 8. *A_F is full row rank. Consequently the matrix*

$$A_F A_F^T$$

is symmetric positive definite.

Proof. Each row of A_F corresponds to one block equality restricted to the free variables. Since the blocks $\{I^k\}_{k \in K}$ are pairwise disjoint, the rows of A_F have disjoint supports.

Moreover, by feasibility, each block contains at least one free variable, otherwise all variables in that block would be zero and the corresponding equality constraint could not be satisfied. Hence every row of A_F is nonzero.

Therefore the rows of A_F are linearly independent, and A_F has full row rank.

Finally, for every nonzero vector y ,

$$y^\top A_F A_F^\top y = \|A_F^\top y\|^2.$$

Since A_F has full row rank, $A_F^\top y = 0$ implies $y = 0$. Thus

$$y^\top A_F A_F^\top y > 0 \quad \forall y \neq 0,$$

which proves that $A_F A_F^\top$ is symmetric positive definite. $\square$

In our implementation, the system is solved by means of a Cholesky factorization. Once λ is computed, the multipliers associated with the bound constraints are obtained from

$$\mu = 2Q\bar{x} + q + A^\top \lambda.$$

The optimality condition for the bound constraints requires

$$\mu_i \geq 0 \quad \forall i \in B.$$

If this condition holds, then the KKT conditions are satisfied and $\bar{x}$ is an optimal solution of problem (P).

Otherwise, if there exists an index $j \in B$ such that

$$\mu_j < 0,$$

the corresponding constraint is removed from the active set and the algorithm proceeds with a new iteration.

3.6 The proposed active-set algorithm

Algorithm 1. Active-set method for problem (P)

- 1: Choose a feasible starting point x^0
- 2: $t \leftarrow 0$
- 3: Define $B^0 = \{i : x_i^0 = 0\}$
- 4: Define $F^0 = \{1, \dots, n\} \setminus B^0$
- 5: **while** true **do**
- 6: Formulate the subproblem in the free variables

$$\min x_F^T Q_{FF} x_F + q_F^T x_F$$

$$\text{s.t.} \quad \sum_{i \in I^k \cap F^t} x_i = 1 \quad \forall k \in K$$

- 7: Apply the null-space parametrization $x_F = x_p + Dz$
- 8: Obtain the reduced quadratic problem in the variable z

$$\min_z \quad \frac{1}{2} z^T H z + g^T z$$

$$\text{where} \quad H = 2D^T Q_{FF} D,$$

$$g = 2D^T Q_{FF} x_p + D^T q_F.$$

- 9: Attempt to solve $H z = -g$ by Cholesky factorization
- 10: **if** the factorization succeeds **then**
- 11: Accept the corresponding solution z $\triangleright$ H is Symmetric Positive Definite
- 12: **else**
- 13: Compute the minimum-norm least-squares solution $\bar{z} = -H^+ g$ by means of the SVD
- 14: Compute the residual vector $r = H \bar{z} + g$
- 15: **if** $\|r\| \leq \epsilon$ **then**
- 16: Set $z = \bar{z}$
- 17: **else**
- 18: Set $p = -r$

19: Define the direction in the full space by

$$d_F = Dp, \quad d_B = 0$$

20: Compute

$$\alpha = \min_{i: d_i < 0} \frac{-x_i^t}{d_i}$$

21: Set $x^{t+1} = x^t + \alpha d$

22: Set $B^{t+1} = \{i : x_i^{t+1} = 0\}$

23: Set $F^{t+1} = \{1, \dots, n\} \setminus B^{t+1}$

24: $t \leftarrow t + 1$

25: **continue**

26: **end if**

27: **end if**

28: Reconstruct the candidate point $\bar{x}$:

$$\bar{x}_F = x_p + Dz, \quad \bar{x}_B = 0$$

29: **if** $\bar{x}_i \geq 0 \quad \forall i$ **then**

30: Set $x^{t+1} = \bar{x}$

31: Set $B^{t+1} = \{i : x_i^{t+1} = 0\}$

32: Set $F^{t+1} = \{1, \dots, n\} \setminus B^{t+1}$

33: **else**

34: Set $d = \bar{x} - x^t$

35: Compute $\alpha = \min_{i: d_i < 0} \frac{-x_i^t}{d_i}$

36: Set $x^{t+1} = x^t + \alpha d$

37: Set $B^{t+1} = \{i : x_i^{t+1} = 0\}$

38: Set $F^{t+1} = \{1, \dots, n\} \setminus B^{t+1}$

39: $t \leftarrow t + 1$

40: **continue**

41: **end if**

42: Compute λ solving

$$(A_{F^{t+1}} A_{F^{t+1}}^\top) \lambda = -A_{F^{t+1}} (2Qx^{t+1} + q)_{F^{t+1}}$$

by Cholesky factorization

```

43:   Compute  $\mu = 2Qx^{t+1} + q + A^\top \lambda$ 
44:   if  $\mu_i \geq 0 \quad \forall i \in B^{t+1}$  then
45:       Stop:  $x^{t+1}$  satisfies the KKT conditions
46:   else
47:        $j = \arg \min_{i \in B^{t+1}} \mu_i$ 
48:       Set  $B^{t+1} = B^{t+1} \setminus \{j\}$ 
49:       Set  $F^{t+1} = \{1, \dots, n\} \setminus B^{t+1}$ 
50:       Set  $x^{t+1} = x^t$ 
51:        $t \leftarrow t + 1$ 
52:   end if
53: end while

```

3.7 Convergence analysis

We briefly discuss the convergence properties of the proposed active-set method.

First, feasibility is preserved at every iteration. The initial point x^0 is chosen feasible, the block equality constraints are explicitly enforced in the reduced subproblem, and the variables in the active set are fixed at zero. Moreover, when the candidate point is not feasible, the algorithm moves along a feasible direction and chooses the largest step length that preserves nonnegativity. Hence every iterate satisfies

$$Ax = b, \quad x \geq 0.$$

At each iteration, the algorithm considers the equality-constrained quadratic subproblem associated with the current active set. If the reduced system

$$Hz = -g$$

is compatible, then the subproblem admits at least one minimizer and a candidate point $\bar{x}$ is reconstructed. If the system is incompatible, then the reduced problem

is unbounded below, and the algorithm computes a feasible descent direction along which it moves until a new bound constraint becomes active.

If, at the candidate point, the multipliers associated with the active bound constraints satisfy

$$\mu_i \geq 0 \quad \forall i \in B^k,$$

then the Karush–Kuhn–Tucker conditions hold and the current point is an optimal solution of problem (P).

Otherwise, at least one active constraint has a negative multiplier, and the corresponding index is removed from the active set. This means that the current active set cannot be optimal, and the algorithm proceeds by solving the subproblem associated with the new working set.

To prove finite termination, we use the standard active-set argument for strictly convex quadratic programming; [2]. At each iteration, exactly one of the following happens:

- the algorithm terminates because the KKT conditions are satisfied;
- a new constraint becomes active during the feasibility step;
- a constraint is removed from the active set because its multiplier is negative.

Whenever the algorithm moves from the current point to a new feasible point during the feasibility step, the step is taken along a descent direction. Therefore, the objective function strictly decreases.

On the other hand, if a constraint is removed from the active set, the current point is not optimal for the original problem, since one of the multipliers violates dual feasibility. After removing that constraint, the algorithm solves a different equality-constrained subproblem. Once the minimizer of that subproblem is reached, either the KKT conditions hold or a new feasible descent step is produced, and in this case the objective function again strictly decreases.

The key observation is that the number of possible active sets is finite. Moreover, once the minimizer associated with a given active set has been computed, the algorithm cannot return to the same active set with the same objective value, because

every change of active set after a minimizer has been reached is followed by a strict decrease of the objective function. Hence cycling is impossible.

Therefore, since only finitely many active sets can be generated and the objective function strictly decreases whenever the method moves from one subproblem minimizer to another active set, the algorithm must terminate in a finite number of iterations. The final point satisfies the KKT conditions and is therefore an optimal solution of problem (P).

3.8 Computational complexity

Let F^k denote the free set at iteration k , and let $m = |F^k|$ be the number of free variables. After the null-space reduction described in Section 3.3, the equality constraints eliminate one variable per block, leading to a reduced problem of dimension $m - |K|$.

The main computational steps of one iteration are the following

- The matrices $H = 2D^T Q_{FF} D$ and $g = 2D^T Q_{FF} x_p + D^T q_F$ must be assembled. The cost of these operations is dominated by matrix–vector and matrix–matrix products and is approximately $O(m^2)$.
- The reduced optimality condition requires solving the linear system

$$Hz = -g.$$

When H is nonsingular, the system is solved using a Cholesky factorization, whose computational cost is

$$O\left(\frac{1}{3}(m - |K|)^3\right). \quad (22)$$

If H is singular and the system admits infinitely many solutions, the solution is computed by means of a Singular Value Decomposition in order to obtain the minimum-norm solution via the pseudoinverse of H . The computational cost of this operation is

$$O((m - |K|)^3). \quad (23)$$

If instead the reduced problem is unbounded below a descent direction is computed by finding a vector in the null space of H , which is again obtained via SVD. The direction in the original space is then computed as $d = Dp$, and the step length is determined to maintain feasibility. Also in this case, the computational cost is dominated by the SVD computation, and is therefore

$$O((m - |K|)^3). \quad (24)$$

The Cholesky factorization can be applied only when H is positive definite and is computationally cheaper since it exploits symmetry and positive definiteness. When H is singular, the SVD must be used instead, which is more expensive but allows us to handle rank-deficient matrices and to determine whether the system $Hx = -g$ is compatible.

- The multipliers λ are obtained by solving the system

$$(A_F A_F^T) \lambda = -A_F(2Q\bar{x} + q)_F.$$

The dimension of the system is $|K|$, and its computational cost is

$$O(|K|^3). \quad (25)$$

Therefore, the dominant computational cost of each iteration of the active-set algorithm is the solution of the reduced linear system, which scales cubically with the number of independent variables.

At each iteration the active set is updated, either by adding a new active constraint or by removing one whose multiplier is negative.

In each partition of the variables the equality constraint

$$\sum_{i \in I^k} x_i = 1$$

must hold. As a consequence, not all variables in the same block can be active simultaneously, since at least one variable per block must remain free.

Hence, if the block I^k contains $|I^k|$ variables, the number of admissible active sets in that block is equal to the number of proper subsets of I^k , namely

$$2^{|I^k|} - 1.$$

Since the blocks are independent, the total number of admissible active sets is bounded by

$$\prod_{k \in K} \left(2^{|I^k|} - 1 \right). \quad (26)$$

This number also represents an upper bound on the number of iterations of the active-set method, since at each iteration the algorithm moves from one active set to another.

Therefore the worst case computational cost of the algorithm is

$$O\left(\prod_{k \in K} \left(2^{|I^k|} - 1 \right) (m - |K|)^3\right). \quad (27)$$

Thus, although the cost of a single iteration is dominated by the solution of a linear system and scales cubically with the number of independent variables, the overall worst-case complexity of the algorithm is exponential. This is due to the potentially combinatorial number of admissible active sets that may be explored before reaching optimality.

4 Experimental Results

In this chapter, we present the numerical experiments performed to evaluate the behaviour of the proposed active-set method. The goal of the experimental analysis is twofold. First, we compare our implementation, denoted by A1, with an off-the-shelf quadratic programming solver, denoted by A2, in order to assess both accuracy and computational efficiency. Second, we study how the method behaves under different problem characteristics, such as increasing problem dimension, different conditioning levels of the matrix Q , positive definite and positive semidefinite matrices, and different sparsity patterns.

For each experiment, we report the execution time, the number of iterations, the equality residual, and, when available, the objective gap with respect to the reference solution. These quantities allow us to evaluate not only the speed of the algorithm, but also its numerical reliability and the quality of the computed solution.

All experiments were performed in MATLAB on a machine equipped with an Apple M5 processor. In particular, the generated instances were solved by both A1 and A2, and the reported values correspond to the same test problems.

4.1 Off-the-shelf solver

In order to assess the quality of the proposed algorithm (A1), we compare it with a general-purpose quadratic programming solver, referred to in the project statement as approach (A2). The goal of this comparison is to provide a meaningful baseline against which the correctness and computational behaviour of our implementation can be evaluated.

For this purpose, we implemented a wrapper around MATLAB’s quadprog routine in the file. More precisely, we use quadprog with the active-set option, so that the comparison is as consistent as possible with the nature of our method. This solution allows us to compare two approaches based on the same general optimization paradigm, rather than contrasting our implementation with a solver based on a completely different strategy.

This comparison is particularly useful for two reasons. First, it allows us to verify that our implementation returns solutions and objective values consistent with those produced by a well-established general-purpose solver. Second, it provides a reference for evaluating the computational trade-off between a solver specifically designed for the structure of problem (P) and a standard optimization tool that treats the problem in a more generic way.

In the following experimental sections, we therefore compare A1 and A2 on the same family of instances, focusing on solution quality, robustness, and execution time.

4.2 Scalability with respect to the problem dimension

The first experiment is devoted to evaluating the scalability of the proposed active-set method when the dimension of the problem increases. In this test, we vary the number of variables n , while keeping the structure of the problem comparable across the different instances. More precisely, the number of blocks is chosen as $K = 0.2n$, so that the average size of each simplex remains approximately constant. The matrix Q is chosen to be symmetric positive definite, with fixed condition number, in order to isolate the effect of the dimension from other numerical factors.

Table 4.1 reports the average execution time, the number of iterations, the equality residual and the objective gap between the proposed method A1 and the off-the-shelf solver A2. The corresponding plots in Figures 4.1, 4.2 and 4.3 provide a visual comparison of the execution time, the number of iterations and the equality residuals as n increases.

n	K	Time		Iter		EqRes		ObjGap
		A1	A2	A1	A2	A1	A2	
10	2	0.0063	0.0262	5	10	0.00e+00	3.44e-16	4.00e-16
50	10	0.0070	0.0043	20	49	0.00e+00	7.66e-16	9.77e-16
100	20	0.0066	0.0060	36	100	0.00e+00	1.07e-15	2.49e-15
200	40	0.0236	0.0200	78	208	0.00e+00	1.52e-15	4.26e-15
500	100	0.1439	0.3507	193	507	0.00e+00	2.54e-15	1.14e-14
1000	200	0.9613	3.7442	376	995	3.33e-17	3.85e-15	1.95e-03
2000	400	4.2680	38.6882	758	1000	7.77e-17	-	-

Table 4.1: Scalability results obtained by increasing the problem dimension n .

The results show that, for small instances, the two methods have comparable execution times. In particular, for $n = 10$, both solvers require approximately the same time. For intermediate sizes, A2 is initially competitive and slightly faster for some instances, which is expected since quadprog is a highly optimized general-

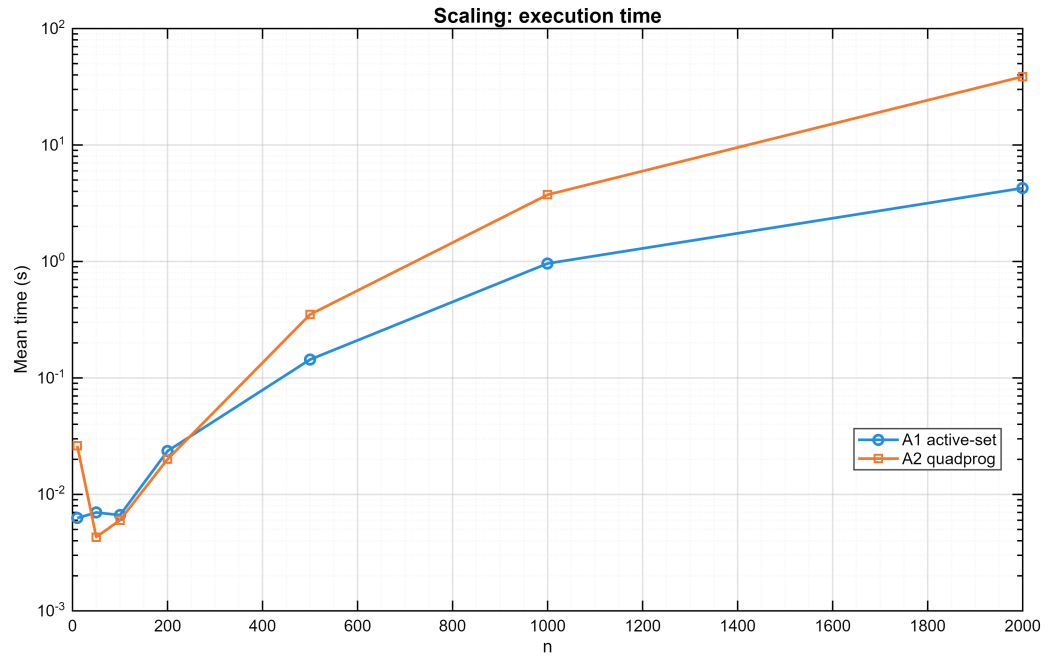


Figure 4.1: Execution time of A1 and A2 as the problem dimension n increases.

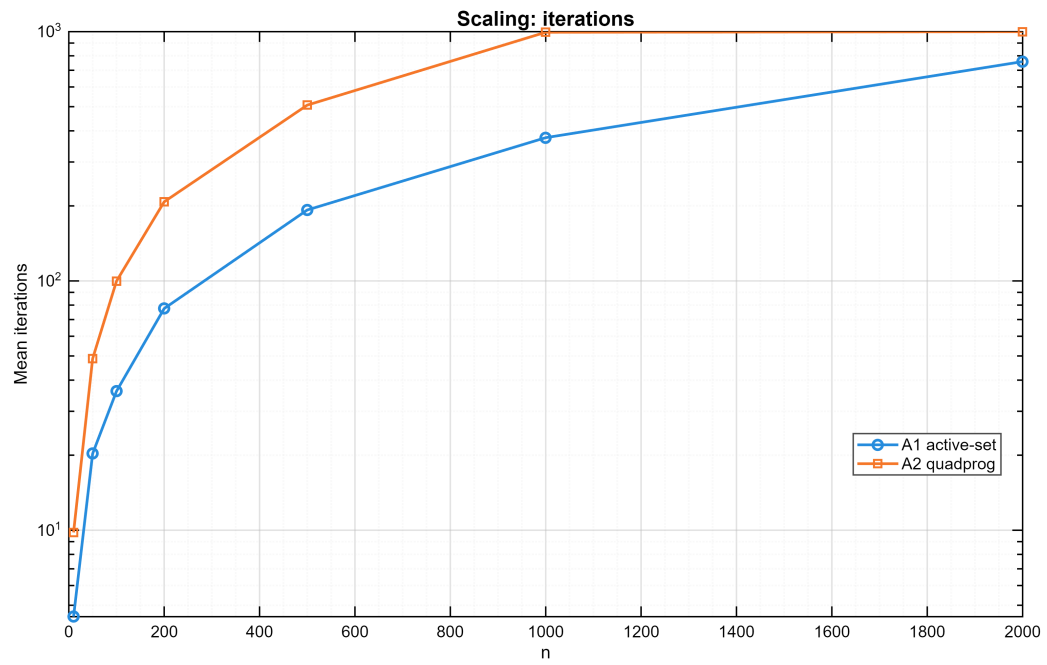


Figure 4.2: Number of iterations of A1 and A2 as the problem dimension n increases.

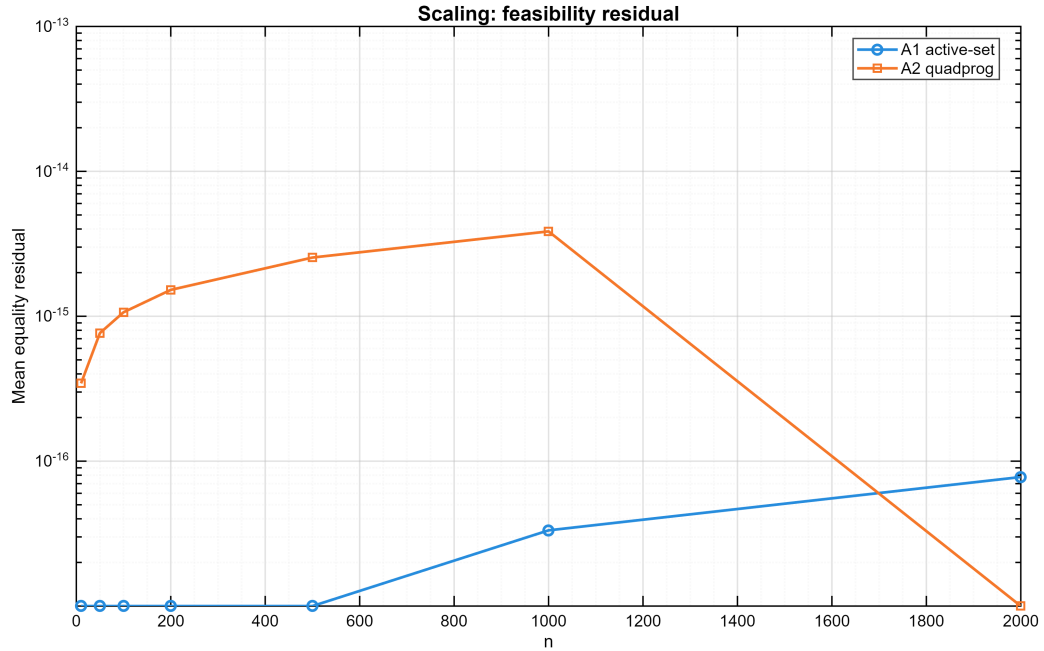


Figure 4.3: Equality constraint residuals of A1 and A2 as the problem dimension n increases.

purpose solver. However, as the dimension increases, the advantage of exploiting the specific structure of the problem becomes more evident. Starting from $n = 500$, A1 becomes significantly faster than A2. For instance, when $n = 1000$, A1 requires about 0.63 seconds, whereas A2 requires about 3.65 seconds. The difference becomes even more pronounced for $n = 2000$, where A1 terminates in about 3.23 seconds, while A2 requires more than 37 seconds.

A similar behaviour can be observed in the iteration plot. The number of iterations of A1 increases with the size of the problem, but it remains controlled and grows in a relatively regular way. On the other hand, A2 requires a larger number of iterations and reaches the maximum iteration limit in the largest instances. This suggests that, although both methods are active-set methods, the proposed implementation benefits from the special block-simplex structure of the constraints.

Although the theoretical worst-case complexity of the active-set method is exponential in the number of admissible active sets, this worst-case behaviour is not observed in the scalability experiment. In fact, the number of iterations of A1 in-

creases with the dimension of the problem, but remains relatively controlled with respect to the combinatorial upper bound. This indicates that, for the tested instances, the proposed method explores only a small fraction of the active sets that are theoretically possible before reaching optimality. Hence, while the worst-case complexity remains exponential, the practical behaviour observed in these experiments is significantly more favourable.

From the point of view of accuracy, the equality residuals remain close to machine precision for A1 across all tested dimensions, confirming that feasibility is preserved with high numerical accuracy. This behaviour is also visible in Figure 4.3, where the residuals of A1 stay very small even for the largest instances. The objective gap is also very small for small and medium-size instances. For larger dimensions, the comparison with A2 becomes less informative whenever the off-the-shelf solver does not fully converge within the prescribed iteration limit. Overall, these results indicate that the proposed method is competitive on small instances and becomes increasingly advantageous as the problem dimension grows.

4.3 Conditioning robustness

The second experiment is designed to evaluate the robustness of the proposed active-set method with respect to the conditioning of the quadratic term. In this test, the dimension of the problem is kept fixed, with $n = 300$ and $K = 60$, while the target condition number of the matrix Q is progressively increased from 10^2 to 10^{12} . In this way, we can study how the two solvers behave when the quadratic objective becomes increasingly ill-conditioned, without changing the size or the block structure of the feasible set.

Table 4.2 reports the average execution time, the number of iterations, the equality residual and the objective gap obtained by A1 and A2 for the different conditioning levels. The corresponding plots in Figures 4.4, 4.5 and 4.6 show the same behaviour visually.

The results show that the proposed method is very robust with respect to the

n	K	$\kappa(Q)$	Time		Iter		EqRes		ObjGap
			A1	A2	A1	A2	A1	A2	
300	60	1.00e+02	0.0546	0.0684	119	310	1.11e-17	2.36e-15	7.82e-15
300	60	1.00e+04	0.0485	0.0706	108	324	0.00e+00	2.53e-15	4.97e-15
300	60	1.00e+06	0.0456	0.0675	103	323	1.11e-17	2.07e-15	4.26e-15
300	60	1.00e+08	0.0458	0.0681	103	325	0.00e+00	2.36e-15	4.97e-15
300	60	1.00e+10	0.0440	0.0686	100	327	0.00e+00	3.02e-15	7.11e-15
300	60	1.00e+12	0.0446	0.0710	101	329	0.00e+00	2.63e-15	7.82e-15

Table 4.2: Conditioning robustness results obtained by increasing the target condition number of Q .

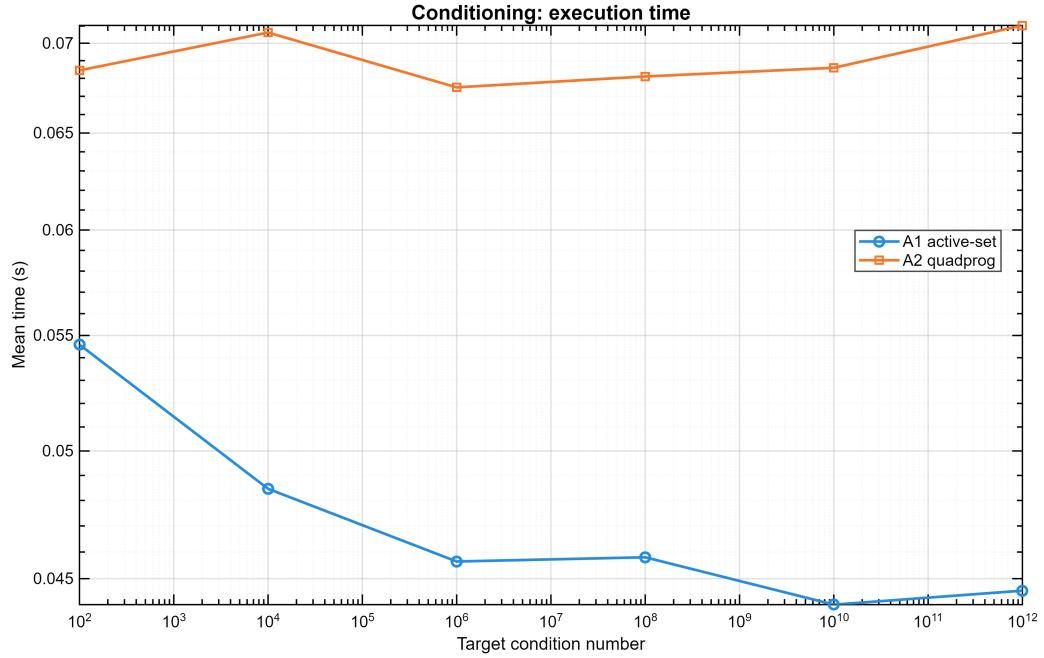


Figure 4.4: Execution time of A1 and A2 as the target condition number of Q increases.

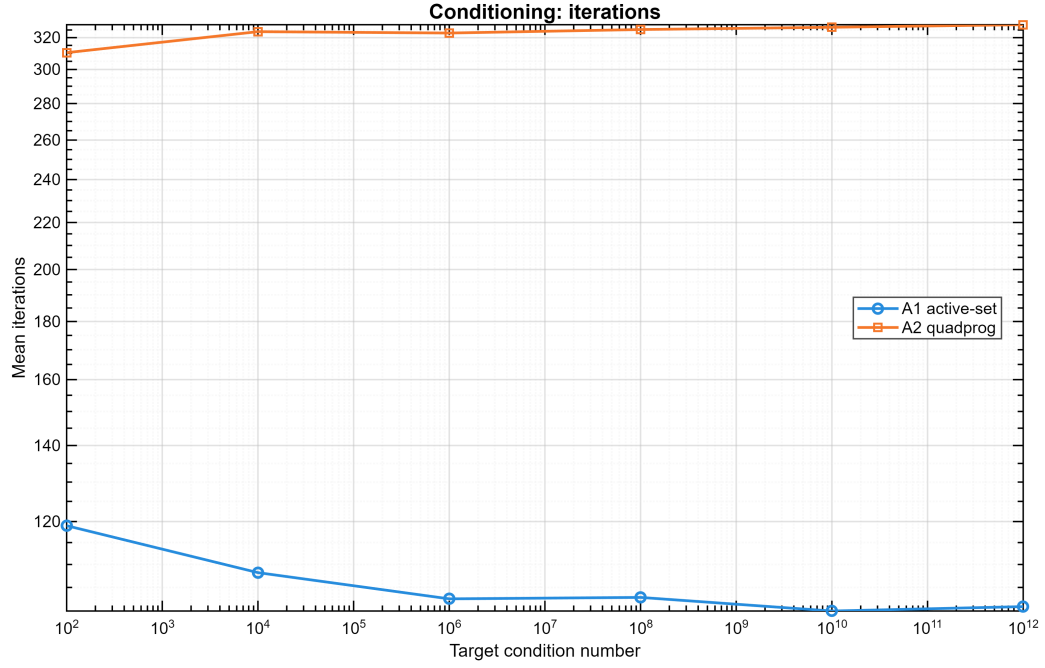


Figure 4.5: Number of iterations of A1 and A2 as the target condition number of Q increases.

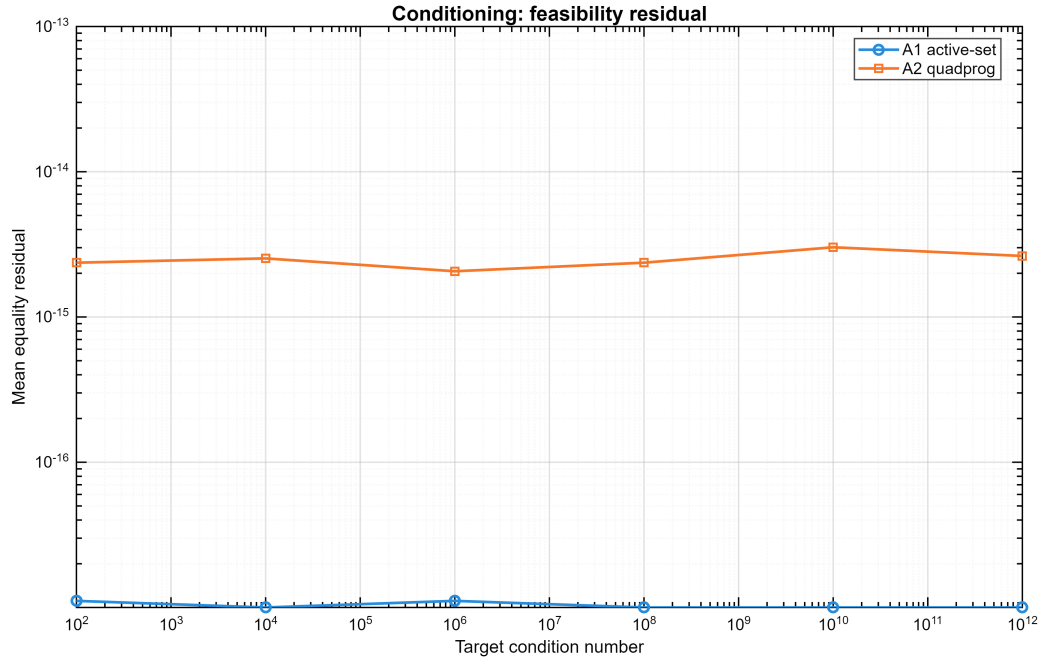


Figure 4.6: Equality constraint residuals of A1 and A2 as the target condition number of Q increases.

conditioning of Q . The execution time of A1 remains almost constant across all tested values of the condition number and is always lower than the execution time of A2. In particular, A1 requires about 0.05 seconds for $\kappa(Q) = 10^2$ and remains around 0.04–0.05 seconds even when the target condition number reaches 10^{12} . By contrast, A2 remains around 0.07 seconds and does not show any improvement as the conditioning changes.

The iteration plot confirms the same behaviour. The number of iterations required by A1 remains significantly smaller than that of A2 for all conditioning levels. Moreover, the number of iterations of A1 slightly decreases as the target condition number increases, while A2 remains almost constant and requires more than 300 iterations in all cases. This indicates that the proposed active-set strategy is not negatively affected by the ill-conditioning introduced in this experiment.

From the point of view of numerical accuracy, both methods preserve feasibility with very small equality residuals. However, A1 consistently achieves residuals close to machine precision, while A2 remains around 10^{-15} . The objective gap is also very small in all cases, confirming that both solvers compute essentially the same optimal value. Overall, this experiment suggests that the proposed method is not only faster than the reference solver on these instances, but also stable when the quadratic term becomes increasingly ill-conditioned.

4.4 SPD vs SPSD

The third experiment is designed to evaluate the behaviour of the proposed active-set method when passing from strictly positive definite matrices to positive semidefinite ones. In this test, the dimension of the problem and the number of blocks are kept fixed, while the kernel dimension of the matrix Q is progressively increased. In particular, the case with zero kernel dimension corresponds to an SPD matrix, whereas positive kernel dimensions correspond to SPSD matrices.

Table 4.3 reports the average execution time, the number of iterations, the equality residual and the objective gap obtained by A1 and A2 for the different kernel dimensions. The corresponding plots in Figures 4.7 and 4.8 show the same behaviour

visually.

Kernel dim	Time		Iter		EqRes		ObjGap
	A1	A2	A1	A2	A1	A2	
0	0.0105	0.0077	54	155	0.00e+00	1.63e-15	4.62e-15
20	0.0109	0.0078	55	159	1.11e-17	1.45e-15	2.13e-15
50	0.0098	0.0080	51	159	0.00e+00	1.52e-15	4.62e-15
80	0.0097	0.0080	51	164	0.00e+00	1.58e-15	1.07e-15
120	0.0099	0.0081	51	167	0.00e+00	1.69e-15	1.78e-15

Table 4.3: Results obtained by increasing the kernel dimension of Q , thus moving from the SPD to the SPSD case.

The results show that the proposed method remains stable when passing from the SPD case to increasingly rank-deficient SPSD matrices. The execution time of A1 is almost constant across all tested kernel dimensions, remaining around 10^{-2} seconds. A2 is slightly faster in absolute time on these instances, but the difference remains limited.

A more significant difference emerges in the iteration counts. The number of iterations required by A1 remains around 50–55 for all tested kernel dimensions, whereas A2 requires more than 150 iterations in every case, and the number of iterations tends to increase as the kernel dimension grows. This suggests that the proposed method is able to handle the loss of definiteness without any substantial degradation in its convergence behaviour.

From the point of view of numerical accuracy, A1 preserves feasibility exactly or up to machine precision in all tested instances, while A2 produces equality residuals of the order of 10^{-15} . The objective gap remains extremely small in all cases, confirming that the two solvers compute essentially the same solution value. Overall, these results indicate that the proposed active-set method is robust not only in the strictly positive definite setting, but also when the quadratic term becomes positive semidefinite.

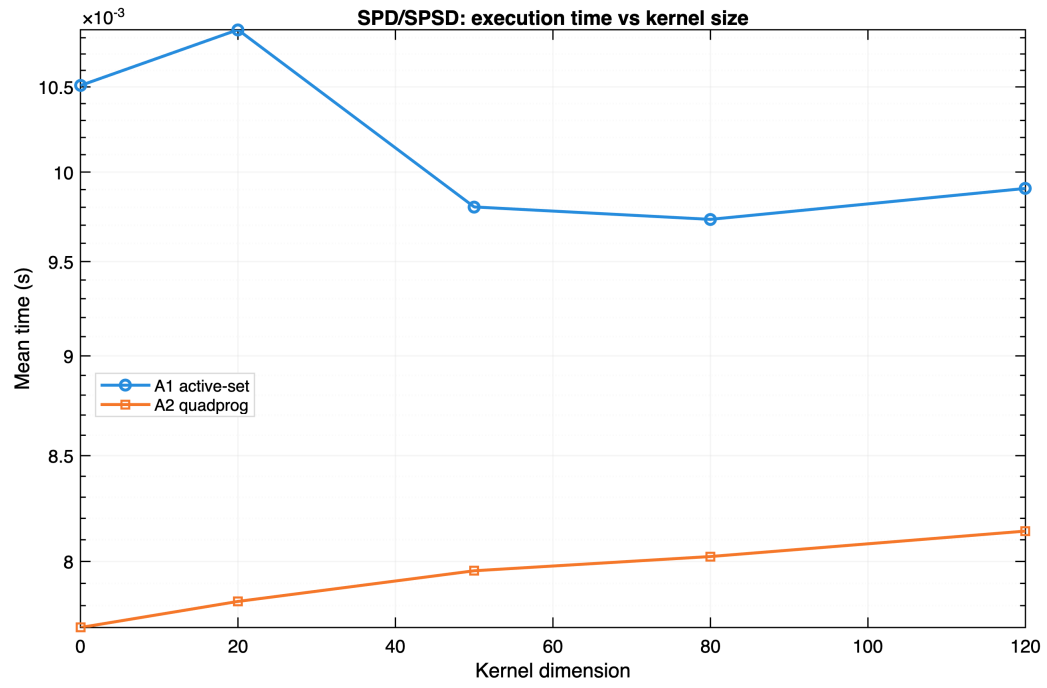


Figure 4.7: Execution time of A1 and A2 as the kernel dimension of Q increases.

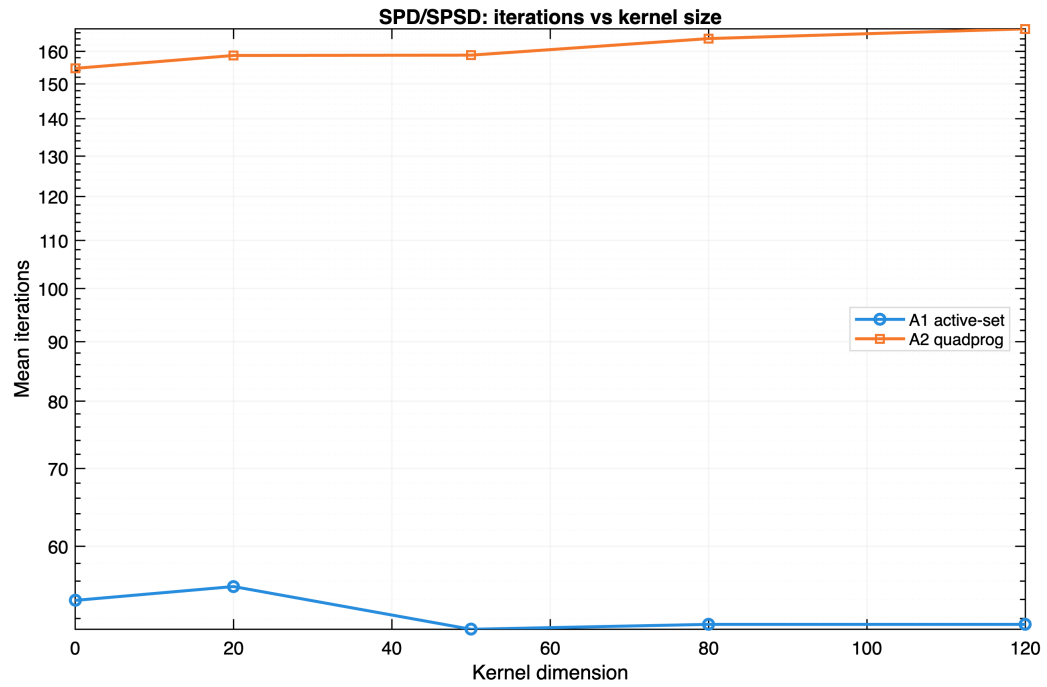


Figure 4.8: Number of iterations of A1 and A2 as the kernel dimension of Q increases.

4.5 Sparsity robustness

The fourth experiment is devoted to evaluating the robustness of the proposed method with respect to the sparsity pattern of the quadratic matrix Q . In this test, the dimension of the problem, the number of blocks, the conditioning level and the matrix type are kept fixed, while the target density of Q is progressively varied. In this way, we can study how the two solvers behave when the quadratic term becomes increasingly sparse.

Table 4.4 reports the average execution time, the number of iterations, the equality residual and the objective gap obtained by A1 and A2 for the different density levels. The corresponding plots in Figures 4.9, 4.10 and 4.11 provide a visual comparison of the behaviour of the two methods as the density changes.

Density	Time		Iter		EqRes		ObjGap
	A1	A2	A1	A2	A1	A2	
1%	0.0418	0.0560	106	223	0.00e+00	2.19e-15	1.21e-14
5%	0.0426	0.0559	107	222	1.11e-17	1.67e-15	5.68e-15
10%	0.0421	0.0558	106	226	0.00e+00	1.82e-15	5.68e-15
20%	0.0433	0.0571	107	223	1.11e-17	1.95e-15	4.97e-15
50%	0.0458	0.0577	108	223	4.44e-17	2.26e-15	8.53e-15
100%	0.0499	0.0580	110	221	2.22e-17	1.90e-15	1.07e-14

Table 4.4: Sparsity robustness results obtained by varying the target density of Q .

The results show that the proposed method is robust with respect to the sparsity pattern of the quadratic matrix. The execution time of A1 is consistently lower than that of A2 for all tested density levels. Moreover, although the execution time of A1 slightly increases as the matrix becomes denser, the variation remains moderate. A2 also shows a mild increase in execution time, but it remains systematically slower than A1 throughout the entire experiment.

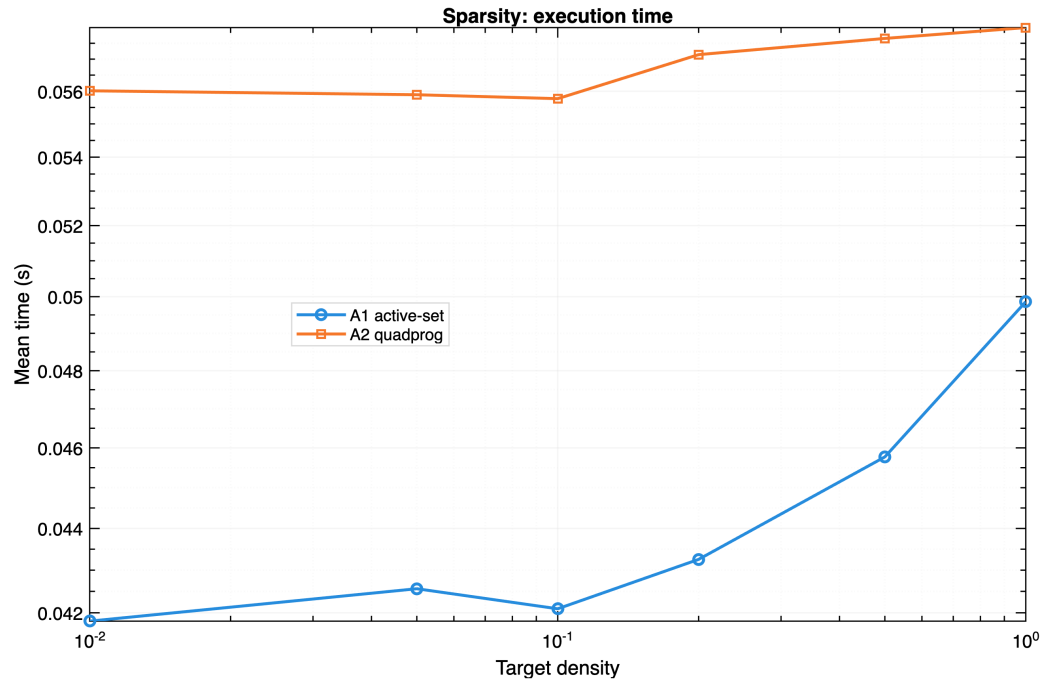


Figure 4.9: Execution time of A1 and A2 as the target density of Q increases.

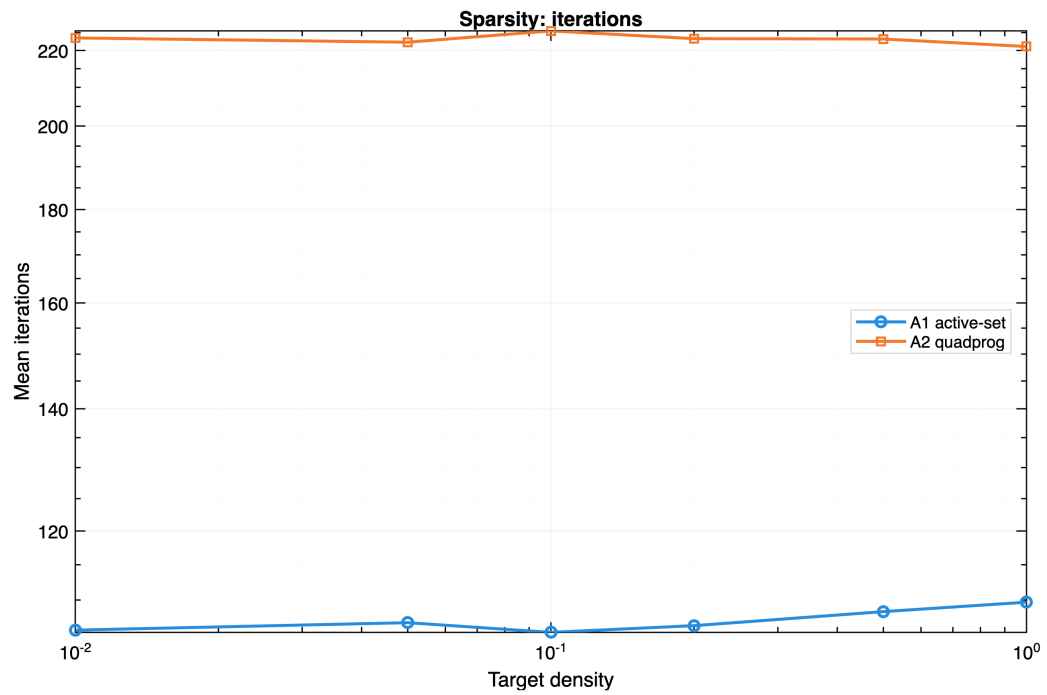


Figure 4.10: Number of iterations of A1 and A2 as the target density of Q increases.

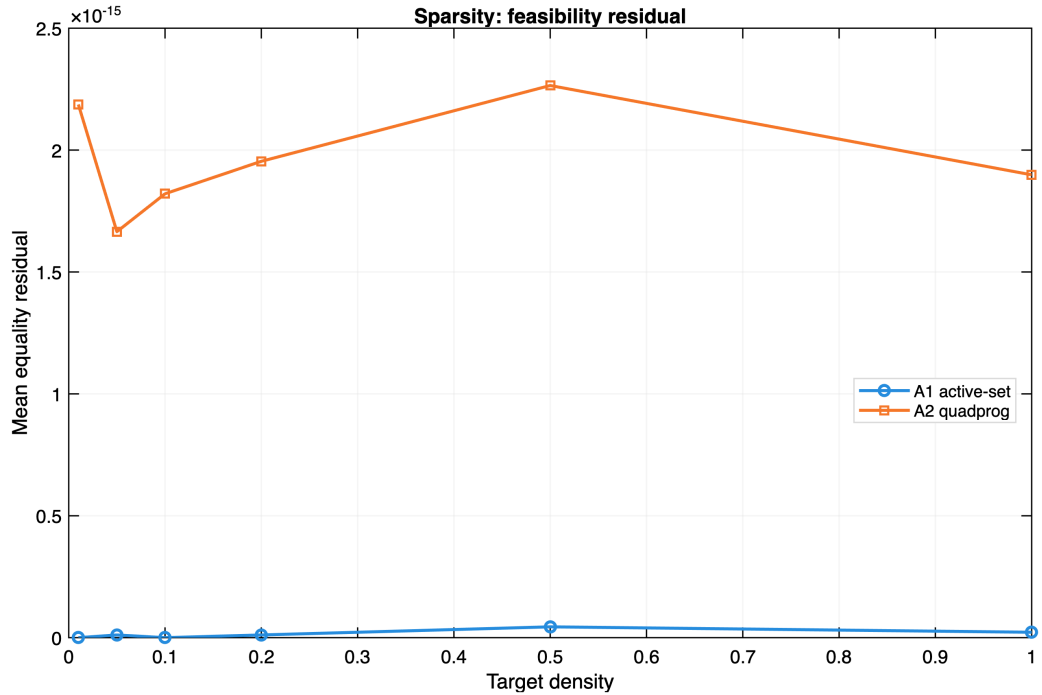


Figure 4.11: Equality constraint residuals of A1 and A2 as the target density of Q increases.

A similar trend can be observed in the iteration counts. The number of iterations required by A1 stays around 106–110 for all tested densities, while A2 requires about 221–225 iterations in every case. Therefore, the proposed method consistently needs less than half the iterations required by the off-the-shelf solver, and this advantage is maintained independently of the sparsity level.

From the point of view of numerical accuracy, both methods preserve feasibility with excellent precision. In particular, A1 again produces equality residuals that are either exactly zero or of the order of machine precision, while A2 remains around 10^{-15} . The objective gap is very small for all densities, confirming that the two methods compute essentially the same optimal value. Overall, the experiment indicates that the proposed active-set method is not only faster than the reference solver, but also stable across a broad range of sparsity patterns.

References

- [1] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. URL:
https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
- [2] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd ed. Springer, 2006. URL:
<https://egrcc.github.io/docs/math/numerical-optimization.pdf>.